

Orquestación de CI/CD con Jenkins

Pipelines Centrales, Repositorios, Nexus y Despliegue Continuo

Prof. Alejandro Muñoz Bravo

Ingeniería de Software V • Universidad ICESI — Período 2026-2

Ruta de Aprendizaje de la Sesión

1 **Arquitectura de Jenkins y Pipelines Declarativos**

2 Integración con Repositorios Git y Triggers

3 Gobernanza con Jenkins Shared Libraries

4 Empaquetamiento, Publicación en Nexus y Despliegue

01

Arquitectura de Jenkins

Del script artesanal a la infraestructura de orquestación escalable

El Orquestador en la Cadena de Valor DevOps

En la entrega continua de software moderno:

- **Motor de automatización:** Coordina la ejecución desatendida de cada etapa del ciclo de vida del software.
- **Desacoplamiento de entornos:** Separa las fases de construcción, prueba, publicación y despliegue del entorno del desarrollador.
- **Visibilidad y trazabilidad:** Centraliza logs, métricas de ejecución, estados de build e historial de cambios.
- **Calidad continua:** Implementa *Quality Gates* automatizados que detienen el avance ante cualquier anomalía.

Principio Clave

Jenkins no compila ni prueba directamente: **orquesta** herramientas especializadas (Git, Maven, Docker, Nexus) garantizando repetibilidad y orden.

Arquitectura Distribuida: Controller y Agents

Jenkins Controller (Master)

Responsabilidades principales:

- Servir la interfaz web y APIs REST.
- Programar y despachar tareas (*jobs*).
- Gestionar credenciales y seguridad RBAC.
- Almacenar el historial de ejecuciones.

Regla de oro: El Controller **nunca** debe ejecutar tareas pesadas de compilación o ejecución de contenedores.

Jenkins Agents (Workers/Nodes)

Ejecución de carga de trabajo:

- Máquinas virtuales, hosts dedicados o contenedores Docker efímeros.
- Reciben instrucciones del Controller por SSH o inbound TCP (50000).
- Proporcionan las herramientas de build (JDK, Node.js, Docker CLI).
- Aislar dependencias entre diferentes proyectos sin colisiones.

Evolución: De Freestyle Jobs a Pipeline as Code

El Problema de los Freestyle Jobs

- Configuración manual dispersa en la UI web.
- Sin control de versiones de los cambios en el pipeline.
- Imposible auditar quién modificó un paso o comando.
- Difícil replicar pipelines entre múltiples proyectos.

Pipeline as Code (Jenkinsfile)

- El flujo de entrega se versiona en el repositorio Git junto al código.
- Evoluciona mediante Pull Requests y revisiones de pares.
- Permite recrear toda la automatización ante un desastre.
- Ejecución idéntica en ramas de desarrollo y producción.

Anatomía de un Declarative Pipeline (Jenkinsfile)

```
pipeline {
  agent any
  tools { maven 'Maven-3.9' }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Test & Build') {
      steps {
        sh 'mvn clean verify'
      }
    }
  }
  post {
    success { echo 'Build exitoso!' }
    failure { echo 'Alerta: Build fallido' }
  }
}
```

Estructura Declarativa

- `pipeline {...}`: Bloque raíz obligatorio.
- `agent`: Define dónde corre (agente Docker, label o nodo).
- `stages`: Secuencia ordenada de etapas del ciclo de vida.
- `steps`: Comandos atómicos ejecutados en cada etapa (`sh`, `echo`).
- `post`: Acciones condicionales ejecutadas tras finalizar (`always`, `success`, `failure`).

Ruta de Aprendizaje de la Sesión

- 1 Arquitectura de Jenkins y Pipelines Declarativos
- 2 Integración con Repositorios Git y Triggers**
- 3 Gobernanza con Jenkins Shared Libraries
- 4 Empaquetamiento, Publicación en Nexus y Despliegue

02

Integración con Git

Disparadores automáticos, multirama y gestión segura de secretos

Mecanismos de Disparo del Pipeline

Polling SCM vs. Cron (Tradicional)

- **Polling SCM:** Jenkins consulta periódicamente al repo Git (ej. cada 5 min).
- **Desventaja:** Sobrecarga inútil de peticiones y latencia en detectar cambios.
- **Cron / Scheduled:** Ejecución a horas fijas (ej. builds nocturnos).

Webhooks en Tiempo Real (Recomendado)

- **Notificación push:** Git notifica a Jenkins de inmediato al ocurrir un `git push` o PR.
- **Latencia cero:** El pipeline inicia segundos después de que el desarrollador sube código.
- **Intranets y Redes Privadas:** Para servidores sin IP pública, GitHub recomienda oficialmente **Smee.io** (relay SSE saliente).
- **Seguridad:** Validado mediante secreto compartido (*HMAC secret token*).

Multibranch Pipelines: Integración Continua Real

Capacidad de Jenkins para descubrir y gestionar ramas automáticamente:

- **Descubrimiento dinámico:** Detecta ramas activas en el repositorio y crea un sub-pipeline por cada una.
- **Validación de Pull Requests:** Ejecuta pruebas y compilación sobre ramas efímeras antes del merge a `main`.
- **Aislamiento:** Evita que un build fallido en una rama de feature interfiera con la rama principal.
- **Limpieza automática:** Elimina pipelines de ramas borradas en Git para no saturar memoria ni disco.

Estrategia de Ramas

Permite implementar **Trunk-Based Development** o **Git-Flow** con feedback inmediato al desarrollador en la misma interfaz de GitHub/GitLab.

Gestión Segura de Secretos (withCredentials)

```
stage('Publish Artifact') {  
    steps {  
        withCredentials([  
            usernamePassword(  
                credentialsId: 'nexus-admin-  
                    creds',  
                usernameVariable: 'NEXUS_USER',  
                passwordVariable: 'NEXUS_PASS',  
            )  
        ]) {  
            sh '''  
                curl -u "${NEXUS_USER}:${NEXUS_PASS}" \  
                    --upload-file target/app.jar \  
                    http://cicd:9081/repository/  
                        maven-releases/...  
            '''  
        }  
    }  
}
```

Seguridad de Credenciales

- **Nunca hardcodear contraseñas** en el Jenkinsfile ni en scripts de Git.
- Jenkins cifra credenciales en su almacén seguro (*Credentials Store*).
- **Enmascaramiento en logs:** Jenkins reemplaza automáticamente cualquier coincidencia de la variable por **** en la consola.
- **Ámbito restringido:** Las variables de entorno solo existen durante el bloque withCredentials.

Ruta de Aprendizaje de la Sesión

- 1 Arquitectura de Jenkins y Pipelines Declarativos
- 2 Integración con Repositorios Git y Triggers
- 3 Gobernanza con Jenkins Shared Libraries**
- 4 Empaquetamiento, Publicación en Nexus y Despliegue

03

Jenkins Shared Libraries

Gobernanza centralizada, reutilización de pipelines y calidad transversal

El Antipatrón del Pipeline Duplicado (Copy-Paste)

Cuando una organización tiene 20 o 50 microservicios:

- **Divergencia de código:** Cada equipo copia el Jenkinsfile de otro y lo modifica a su criterio.
- **Vulnerabilidades de seguridad:** Actualizar una versión de herramienta o credencial requiere cambiar 50 archivos.
- **Falta de estándares:** Unos proyectos ejecutan SonarQube, otros pruebas de integración, y otros se saltan pasos.
- **Alto costo de mantenimiento:** Imposible aplicar mejoras globales de infraestructura de forma atómica.

La Solución

Jenkins Shared Libraries:

Extraer la lógica completa del pipeline a un repositorio Git centralizado, versionado y gobernado por el equipo DevOps.

Estructura Canónica de una Jenkins Shared Library

Árbol de Directorios del Repo Central

```
[repo-shared-library]/
+-- vars/                  % Steps DSL
|  +-- standardPipeline.groovy
|  +-- publishToNexus.groovy
|  +-- deployContainer.groovy
+-- src/                   % Clases OOP
|  +--
org/icesi/devops/NexusClient.groovy
+-- resources/             % Plantillas
+-- docker-compose.tmpl.yml
```

Propósito de cada Carpeta

- `vars/`: Define pasos personalizados ejecutables directamente en el Jenkinsfile como funciones.
- Cada archivo `nombrePaso.groovy` expone un método `def call(Map config)`.
- `src/`: Código Groovy estructurado con paquetes orientados a objetos para lógica compleja y utilidades.
- `resources/`: Ficheros no ejecutables (JSON schemas, plantillas de despliegue, certificados).

Implementación de la Librería Central (vars/standardPipeline.groovy)

```
def call(Map config = [:]) {
    pipeline {
        agent any
        environment {
            APP_NAME      = config.appName ?: 'microservice'
            BUILD_TOOL    = config.buildTool ?: 'maven'
        }
        stages {
            stage('Checkout') { steps { checkout scm } }
            stage('Build & Test') {
                steps {
                    script {
                        if (env.BUILD_TOOL == 'maven') { sh 'mvn clean verify' }
                        else { sh 'npm test' }
                    }
                }
            }
            stage('Publish to Nexus') {
                steps { publishToNexus(appName: env.APP_NAME) }
            }
        }
    }
}
```

Consumo del Pipeline Central en el Repositorio de la App

Jenkinsfile en el Repo del Microservicio

```
// Importa la libreria central
// versionada
@Library('iaslab-shared-library@v1
        .2') _

// Invoca el pipeline estandarizado
standardPipeline(
    appName: 'catalogo-productos',
    buildTool: 'maven',
    enableNexusPublish: true,
    deployEnv: 'staging'
)
```

Beneficios Inmediatos

- El Jenkinsfile del microservicio tiene solo **6 líneas**.
- El desarrollador no necesita ser experto en Jenkins ni en scripts Groovy.
- Seguridad y gobernanza garantizadas: escaneos y validaciones obligatorias inalterables.
- Versionamiento de la librería mediante tags Git (@v1.2).

Ruta de Aprendizaje de la Sesión

- 1 Arquitectura de Jenkins y Pipelines Declarativos
- 2 Integración con Repositorios Git y Triggers
- 3 Gobernanza con Jenkins Shared Libraries
- 4 Empaquetamiento, Publicación en Nexus y Despliegue**

04

Nexus y Despliegue CD

Empaquetamiento determinista, publicación inmutable y entrega continua

BUILD ONCE

Compila una única vez, promueve el mismo artefacto inalterable

“Si un binario o imagen Docker se recompila en cada ambiente, se invalidan todas las pruebas ejecutadas en etapas anteriores. Lo que se probó en CI debe ser exactamente lo que corre en producción.”

Publicación en Sonatype Nexus desde el Pipeline

Artefactos Java / Maven (port 9081)

Repositorios Hosted en Nexus:

- `maven-releases`: Artefactos cerrados e inmutables (versiones semánticas fijas como 1.0.0).
- `maven-snapshots`: Artefactos de desarrollo dinámicos en ramas feature.
- Publicación mediante `mvn deploy` con credenciales seguras de Nexus inyectadas en `settings.xml`.

Imágenes OCI / Docker (port 9080)

Docker Registry Privado:

- Construcción con `docker build -t cicc:9080/app:1.0.0-$BUILD_NUMBER`.
- Etiquetado dual: versión semántica + commit hash exacto de Git.
- Autenticación mediante `docker login cicc:9080`.
- `docker push` al registro central de artefactos.

Despliegue Continuo (CD): De Nexus al Servidor

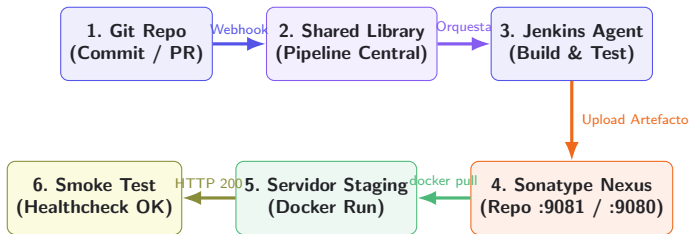
El pipeline ejecuta el paso de entrega sobre el servidor de despliegue:

- **Pull del artefacto publicado:** El servidor objetivo descarga la imagen directamente de Nexus (`docker pull cicd:9080/app:1.0.0`).
- **Cero compilación en runtime:** En el servidor no hay Git, ni compilador, ni Maven; solo el motor de ejecución (Docker runtime).
- **Reemplazo atómico:** Detiene el contenedor previo, levanta el nuevo con las variables de entorno de staging y mapea puertos.
- **Smoke test post-despliegue:** Ejecuta peticiones HTTP inmediatas (`/actuator/health`) para certificar operatividad.

Estrategia de Rollback

Si el healthcheck falla tras el despliegue, el pipeline ejecuta automáticamente `post { failure { rollback() } }`, reactivando la versión previa de Nexus en segundos.

Flujo Integral de Trazabilidad CI/CD en lasLab



Trazabilidad Total: El hash del commit Git se propaga a la versión de la imagen en Nexus y a los logs del contenedor en ejecución.

Preguntas de Reflexión y Debate para la Sesión

Gobernanza y Arquitectura

1. ¿En qué momento una organización debe migrar de pipelines individuales a una Shared Library central?
2. Si la Shared Library central tiene un bug, ¿qué impacto tiene sobre los microservicios y cómo se mitiga?
3. ¿Por qué usar Nexus como proxy intermedio de dependencias externas mejora la resiliencia corporativa?

Operaciones y Seguridad

4. ¿Cuáles son los riesgos de almacenar credenciales SSH con acceso root en el almacén de Jenkins?
5. Si un contenedor falla su healthcheck tras un despliegue CD, ¿cuál es el mecanismo más limpio de recuperación?